CSL559: COMPUTER SYSTEMS DESIGN

Dr. Gagan Raj Gupta



MOTIVATION OF THE COURSE

- Many fundamental ideas concerning computer systems, such as design principles, modularity, naming, abstraction, concurrency, resource allocation, communications, fault tolerance, and atomicity, are common to several of the upper-division electives of the Computer Science and Engineering (CSE) curriculum.
- The objective of this course is to bring those concepts together, holistically and apply them to design of large scale applications



COMMON REQUIREMENTS OF APPS

- Efficient allocation of resources
- Fault tolerance
- Coordination of concurrent activities
- Vast quantities of stored information
- Protection from mistakes and intentional attacks
- Interactions with many people
- Geographically separated but linked data



SYSTEMS

- To develop applications that have these requirements, the designer must look beyond the software and hardware and view the computer system as a whole
- A *system* is a set of interconnected components that has an expected behavior observed at the interface with its environment
  - Described by its boundaries, structure and purpose and expressed in its functioning
- To analyze a system one must establish a point of view to determine which things to consider as components, what the granularity of those components should be, where the boundary of the system lies, and which interfaces between the system and its environment are of interest



SYSTEM DESIGN AND ANALYSIS

- *Computer system i*s a system intended to store, process, or communicate information under automatic control
- We are primarily interested in distributed computer systems:
  - CEPH: Distributed storage
  - NFS/HDFS: Distributed File system
  - Hadoop/Spark: Distributed computing for big-data
  - Condor: Large scale computing
  - Kafka: Distributed Message Bus
  - Interesting Applications: URL Shortener, Dropbox, Chatbot, Messenger, Recommendation Engine, Twitter, Webcrawler, Newsfeed, Yelp, Uber, Ticketmaster, Etherpad …



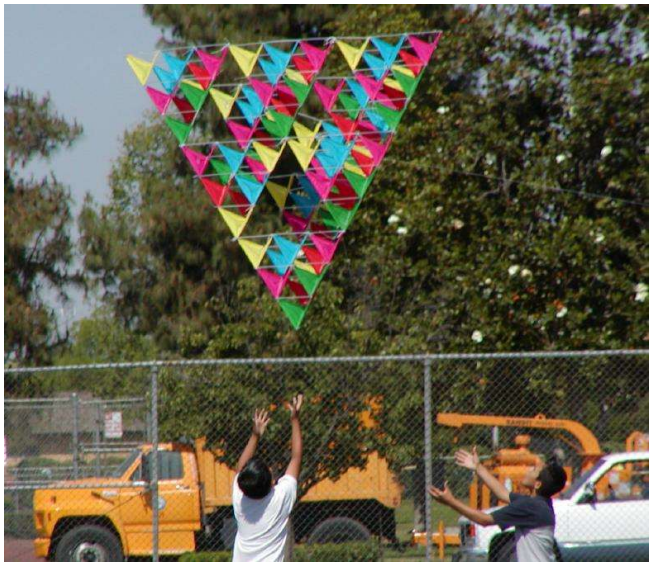
COMMON PROBLEMS OF SYSTEMS

- Emergent properties: *Surprises during integration*
  - An unalterable fact of life: some things turn up only when a system is built.
  - Electric grids, Bridge in London
- Propagation of effects: *Small changes leading to Big Consequences*
  - Change tire size in automobile
  - There are no small changes in a large system
- Incommensurate scaling
  - Different components scale differently
  - Can a mouse be scaled to the size of an elephant?
- Trade-offs
  - Higher clock rate in a circuit
  - Classification (precision vs recall)



WHY AIRPLANES CAN'T FLY?

- The weight of an airplane grows with the third power of its linear dimension, but the lift, which is proportional to surface area, can grow only with the second power. Even if a small plane can be built, a larger one will never get off the ground.
- This line of reasoning was used around 1900 by both physicists and engineers to argue that it was a waste of time to build heavier-than-air machines. Alexander Graham Bell proved that this argument wasn't the whole story by flying box kites in Maine in the summer of 1902.
- In his experiments he attached two box kites side by side, a configuration that doubled the lifting surface area, but also allowed removal of the redundant material and supports where the two kites touched. Thus, the lift-to-weight ratio actually improved as the scale increased. Bell published his results in "The tetrahedral principle in kite structure"



SIGNS OF COMPLEXITY

- **Large number of components.** Sheer size
- **Large number of interconnections.** Even a few components may be interconnected in an unmanageably large number of ways.
- **Many irregularities.** Lack of regularity, as shown by the number of exceptions or by non-repetitive interconnection arrangements, strongly suggests complexity.
- **A long description.** Looking at the best available description of the system one finds that it consists of a long laundry list of properties rather than a short, systematic specification that explains every aspect.
- **A team of designers, implementers, or maintainers.** A complex system's description, construction, or maintenance will require not just technical expertise but also coordination and communication across a team.



CASCADING AND INTERACTING REQUIREMENTS

- *Adding a requirement increases complexity out of proportion*
- Telecom: call waiting, call return, call forwarding, originating and terminating call blocking, reverse billing, caller ID, caller ID blocking, anonymous call, rejection, do not disturb, vacation protection
- For example, so-called 900 numbers have the feature called reverse billing—the called party can place a charge on the caller's bill.
  - Alice has a feature that blocks outgoing calls to reverse billing numbers
  - Alice calls Bob, whose phone is forwarded to a 900 number
  - Should the call go through, and if so, which party should pay for it, Bob or Alice?
  - There are three interacting features, and at least four different possibilities: block the call, allow the call and charge it to Bob, ring Bob's phone, or add yet another feature that (for a monthly fee) lets Bob choose the outcome.

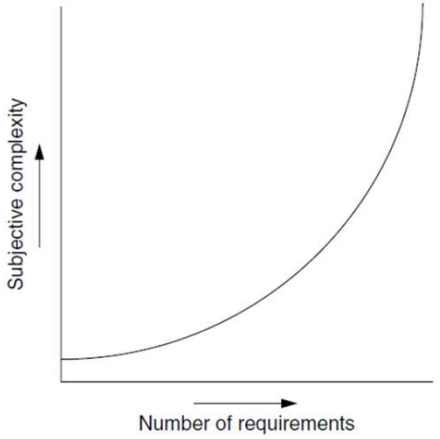


FIGURE 1.1
The principle of escalating complexity.

Avoid excessive generality

If it is good for everything, it is good for nothing.

MAINTAIN HIGH UTILIZATION

- The more one improves some measure of goodness, the more effort the next improvement will require
- A rarely used street intersection requires no traffic control beyond a rule that the car on the right has the right-of-way.
- As usage increases, one must apply progressively more complex measures: stop signs, then traffic lights, then marked turning lanes with multiphase lights, then vehicle sensors to control the lights.
- As traffic in and out of an airport nears the airport's capacity, measures such as stacking planes, holding them on the ground at distant airports, or coordinated scheduling among several airlines must be taken.

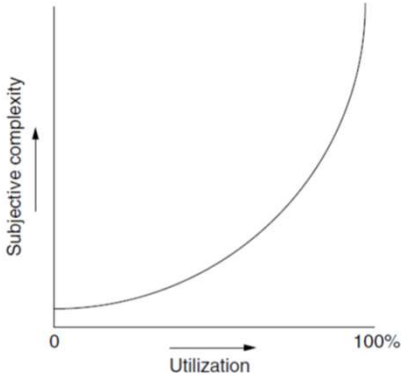


FIGURE 1.2

An example of diminishing returns: complexity grows with increasing utilization.

LIMITING COMPLEXITY

- **Modularity:** divide-and-conquer technique
  - Analyze or design the system as a collection of interacting subsystems, called *modules*
  - Best divisions follow natural boundaries, fewer interactions, less propagation of effects
  - A module can be improved and allows for incremental improvement of the system
  - Debug time grows proportional to the size of the code
- **Abstraction:**
  - Abstraction is separation of interface from internals, of specification from implementation
  - Modules treat each other entirely on the basis of external specification, without knowing what goes inside.
  - Robustness: *Be tolerant of inputs and strict on outputs.*
  - The safety margin principle: Keep track of the distance to the cliff, or you may fall over the edge.
- **Layering:** Eg. layers in networks, computers (Gates->Processor->OS->App)
  - Modules in a layer can only interact with its peers and its upper and lower layers
- **Hierarchy:** Tree like structure, as in organizations
  - Reduces the number of potential interactions among modules from square-law to linear.



NAMING AND BINDING

- Ok, we have divided the problem and placed the modules in relation to each other.
- How do we connect the modules? Primary connection method is through naming.
- Names allow postponing of decisions, easy replacement of one module with a better one, and sharing of modules. Software uses names in an obvious way.
  - Less obviously, hardware modules connected to a bus also use names for interconnection—addresses, including bus addresses, are a kind of name.
- Allows one to design a module as if a feature of another module exists, even if that feature has not yet been implemented, and it also makes it easy to later choose a different implementation.
- By the time the feature is actually invoked, the name must, of course, be bound to a real implementation of the other module.
- Using a name to delay or allow changing a binding is called *indirection*, and the design principle **Decouple modules with indirection:** *Indirection supports replaceability.*



PERVASIVE IMPORTANCE OF MODULARITY

- *Abstractions and layering* are particular ways to build on modularity
- *Naming* is a fundamental mechanism for interconnecting and replacing modules
- *Clients and services* and *virtualization* are two ways of enforcing modularity
- *Networks* are built on a foundation of modularity
- In *fault tolerance*, the module is the unit that limits the extent of failure
- *Atomicity* is an exceptionally robust form of modularity that the designer can exploit to obtain *consistency*
- *Protection of information* involves further strengthening of modular walls



PHYSICAL SYSTEMS VS. COMPUTER SYS

- There are at least two significant ways in which computer systems differ from every other kind of system with which designers have experience:
- The complexity of a computer system is not limited by physical laws.
  - Noise, for example is a limit to the complexity of analog systems
  - Digital systems don't accumulate noise, the outputs stay within narrow specs.
  - Software has no limits beyond the availability of memory to store it and processors to execute
- The rate of change of computer system technology is unprecedented
  - Completely unrealistic ideas a year or two ago can become mainstream in no time, so the computer system designer almost always ends up designing something that is significantly different from the previous one
  - Rapidly improving technology means that brute-force solutions (buy more memory, wait for a faster processor, use a simpler algorithm) are often the right approach in computer systems, whereas in other systems they may be unthinkable.
- These two differences have an enormous impact on complexity and ways of coping with it



OTHER DIFFERENCES

- **High Variability:** in job sizes (service requirements), with squared coefficients of variation upward of 100, very different from manufacturing workloads.
- **Pre-emptible and Time-shared:** Processor-Sharing or more advanced preemptive policies like Shortest-Remaining-Processing-Time, are very much at the heart of computer systems.
- Performance metrics may also be different, e.g., **power usage**
- Closed-loop architectures, in which new jobs are not created until existing jobs complete, and where the performance goal is to maximize throughput, are very common in computer systems
- Complex interactions that occur in disks, networking protocols, databases, memory controllers



ITERATION (AGILE)

- Start by building a simple, working system that meets only a modest subset of the requirements and then evolve that system in small steps to gradually encompass more and more of the full set of requirements
  - Small steps can help reduce the risk that complexity will overwhelm a system design.
  - Having a working system available at all times helps provide assurance that something can be built and provides on-going experience with the current technology ground rules as well as an opportunity to discover and fix bugs
  - Adjustments for technology changes that arrive during the system development are easier to incorporate as part of one or more of the iterations
- **Design for iteration:**
  - You won't get it right the first time, so make it easy to change



ITERATION

Take small steps: allow for discovery of mistakes, bad ideas (CI/CD)

Don't rush: Individual steps must be well planned,

Plan for feedback: design feedback paths and positive incentives for feedback

Study failures: Incentives for feedback about failures, Don't ignore unexplained behavior

Keep digging

Complex systems fail for complex reasons



ITERATION (OBSTACLES)

- The main obstacle is that as a design evolves through a series of iterations, a risk of losing conceptual integrity arises
  - Overall plan for the initial, simplest version of the system must accommodate all of the iterations needed to reach the final version (thus the need for foresight)
  - Someone must constantly be on guard to make sure that the overall design rationale remains clear despite changes made during iteration
- A longer-term risk of iteration sometimes shows up when the initial design is both simple and successful
  - Success can lead designers to be overconfident and to be too ambitious on a later iteration
  - Technology has improved in the time since deployment of the initial version of the system and feedback has suggested lots of new features
  - Each suggested feature looks straightforward by itself, and it is difficult to judge how they might interact
  - The result is often a disastrous overreaching and consequent failure that is so common that it has a name: the *second-system effect*



KEEP IT SIMPLE

- Problems with this obvious advice
  - previous systems give a taste of how great things could be if more features were added
  - technology has improved so much that cost and performance are not constraints
  - each of the suggested new features has been successfully demonstrated somewhere
  - none of the exceptions or complications seems by itself to be especially hard to deal with
  - there is fear that a competitor will market a system that has even more features
  - among system designers, arrogance, pride, and overconfidence are more common than clear awareness of the dangers of complexity
- Computer system designer's most potent weapon against complexity is the ability to say, "No. This will make it too complicated."

KISS: Keep it Simple, Stupid
Fools ignore complexity. Pragmatists suffer it. Some can avoid it. Geniuses remove it.
Everything should be made as simple as possible, but no simpler - Einstein



COURSE PLAN

- Teaching
  - 2 Lectures a Week
  - 1 Tutorial Session
  - Practice Programming Problems/Case Studies every week
  - Lab exercises
- Evaluation

Marks	Grades
	A+
	A
	A-
	B
	B-
	C
	C-
	D
	F



URL SHORTENING SERVICE

- Benefits: space savings, less errors, user monitoring, ad services
- Functional Requirements
  - Size of the short-url
  - Redirection to the original url
  - Link expiry
- Non-Functional Requirements
  - High-availability
  - Minimal latency
  - Short-urls must not be predictable (??)
- Extended Requirements
  - Analytics
  - Access via API



CAPACITY ESTIMATION

- Read: Write ratio = 100:1
- Assume we have 500 M new URL shortening per month
  - Number of re-directions:
- Queries per second:
- URL redirections per second:
- Storage estimate for 5 years:
- Bandwidth estimates
- Memory estimate: 20% cache



CAPACITY ESTIMATES: URL SHORTENING

Types of URLs	Time estimates
New URLs	200/s
URL redirections	20K/s
Incoming data	100KB/s
Outgoing data	10 MB/s
Storage for 5 years	15 TB
Memory for cache	170 GB



- https://explore.skillbuilder.aws/learn/course/external/view/elearning/11458/aws-cloud-quest-cloud-practitionerLinks to an external site.
- https://www.coursera.org/learn/aws-cloud-practitioner-essentialsLinks to an external site.
- https://aws.amazon.com/getting-started/hands-onLinks to an external site.

